

Manual de Funcionamiento del Sistema

1. Descripcion general

El sistema es una aplicacion web fullstack para la gestion de tareas colaborativas.

Esta compuesto por:

- `Frontend` en Angular (interfaz de usuario).
- `Backend` en Node.js + Express (API REST).
- `Base de datos` PostgreSQL gestionada con Prisma ORM.

Su objetivo es permitir a los usuarios autenticados crear, consultar, actualizar y eliminar tareas, ademas de cambiar su estado dentro de un tablero visual.

2. Arquitectura y componentes

2.1 Frontend (Angular)

Responsabilidades principales:

- Renderizar vistas de autenticacion y modulo de tareas.
- Consumir la API REST mediante servicios Angular.
- Manejar formularios reactivos y validaciones basicas.
- Mostrar feedback visual con modales, confirmaciones y estados de carga.

Componentes clave:

- `auth`: login y registro.
- `layouts`: estructura de navegacion.
- `tasks/index`: tablero Kanban y operaciones CRUD.

Servicios clave:

- `tasks`: consumo de endpoints de tareas.
- `auth`: gestion de sesion y usuario local.
- `users`: carga de usuarios para asignacion.

2.2 Backend (Node.js + Express)

Responsabilidades principales:

- Exponer endpoints REST versionados (`/api/v1`).
- Validar datos de entrada con esquemas.
- Aplicar autentificacion por token (JWT).
- Ejecutar logica de negocio y persistir con Prisma.
- Estandarizar respuestas de exito y error.

Capas principales:

- `routes`: definicion de endpoints.
- `controllers`: logica por caso de uso.
- `middleware`: autentificacion, validacion y manejo de errores.
- `schemas`: reglas de validacion (Zod).
- `config/prisma`: cliente de conexion a base de datos.

2.3 Base de datos (PostgreSQL + Prisma)

Entidades principales:

- `User`
- `Roles`
- `Tasks`
- `Notifications`

La entidad `Tasks` guarda informacion como titulo, descripcion, fecha de vencimiento, usuario creador, usuario asignado y estado.

3. Flujo funcional del sistema

3.1 Autenticacion

1. El usuario inicia sesion desde el frontend.
2. El backend valida credenciales y emite token JWT.
3. El frontend guarda el token y lo envia en cada solicitud protegida.
4. El middleware de autentificacion valida el token antes de procesar endpoints privados.

3.2 Gestion de tareas

Flujo principal:

1. El frontend solicita listado de tareas al backend.
2. El backend filtra tareas relacionadas al usuario autenticado.

3. El usuario puede crear/editar/eliminar tareas desde el tablero.
4. Cada accion se persiste en base de datos y actualiza la interfaz.
5. El cambio de estado puede realizarse por formulario o drag-and-drop en Kanban.

3.3 Notificaciones

Cuando se asigna una tarea o cambia su estado, el backend crea notificaciones asociadas. Estas notificaciones pueden ser consumidas por los modulos de frontend correspondientes.

4. API REST principal

Base URL: `http://localhost:3000/api/v1`

Endpoints de tareas:

- `GET /tasks` - listar tareas.
- `GET /tasks/:taskId` - obtener una tarea.
- `POST /tasks` - crear tarea.
- `PUT /tasks/:taskId` - actualizar tarea.
- `PUT /tasks/:taskId/state` - actualizar estado.
- `DELETE /tasks/:taskId` - eliminar tarea (borrado logico).

5. Validaciones, errores y seguridad

5.1 Validaciones

- Validacion de body con Zod en operaciones de creacion y actualizacion.
- Campos requeridos como titulo y usuario asignado.
- Validacion de estados permitidos de tarea.

5.2 Manejo de errores

- Middleware centralizado para errores.
- Respuestas estandarizadas con codigo HTTP y mensaje.
- Manejo de casos de Prisma (registro no encontrado, restricciones, etc.).

5.3 Seguridad

- Endpoints de tareas protegidos por JWT.
- Verificacion de formato `Bearer token`.
- Politica CORS configurada para origenes permitidos.

6. Funcionamiento con Docker

El proyecto usa `docker-compose` para levantar todo el stack:

- `db` (PostgreSQL)
- `backend`
- `frontend`
- `adminer`

Al iniciar contenedores, el backend ejecuta:

1. Generacion de cliente Prisma.
2. Migraciones.
3. Seed de datos de prueba.

Esto permite tener un entorno funcional de forma rapida y reproducible.

7. Flujo de uso para el usuario final

1. Iniciar sesion en la plataforma.
2. Visualizar tablero de tareas.
3. Crear nueva tarea y asignarla.
4. Editar informacion o mover de estado.
5. Eliminar tareas cuando sea necesario.

8. Conclusiones

El sistema implementa una arquitectura modular y escalable, con separacion de responsabilidades entre frontend, backend y persistencia.

La operacion dockerizada reduce friccion de despliegue y facilita la evaluacion tecnica del proyecto.